KisanSetu — Product Requirements Document (v1.1, 24-hour build)

An AI decision assistant for Indian farmers: one photo or voice note in, one clear action plan out.

Version: 1.1 · **Date:** 17 Oct 2026 · **Status:** Final draft for team review **Team:** Utsav (Agent / GenAI) · Aryan (ML) · Arpit (ML + Frontend + Backend) **Theme:** Agriculture & Rural Development

"KisanSetu" (setu = bridge) is a placeholder name. Nothing in the codebase depends on it.

What changed from v1.0

- **Hackathon is 24 hours.** The cut plan is now the plan (Sections 1, 6, 16, 17). Everything outside it is marked Deferred.
- **One combined backend.** core-api, agent-service and the ML code are deployed as a single FastAPI app, one Docker image, one URL, on Hugging Face Spaces (Sections 9, 14, 20). Folders and ownership stay separate.
- **ml-service is no longer an HTTP service.** It is a Python module called directly. No internal key, no network hop (Sections 8, 9, 12, 13).
- **Models are trained on Colab and loaded from saved files.** Disease CNN as ONNX, price model and other sklearn-family models as joblib, loaded once at startup, stored on Hugging Face Hub and never in git (Sections 7, 15.3, 20).
- Deferred to roadmap: crop recommender, irrigation planner, scheme assistant, notifications, quality grading, offline-lite, daily-plan endpoint.

0. How to use this document

This is the single source of truth. Sections 1–10 say what we build and why. Sections 11–15 say how it is wired. Sections 16–23 say who does what, when, plus UI, deployment, demo and risks.

- If this document and a chat message disagree, the document wins. Change the document first, then the code.
- **P0** must work in the demo. **P1** is built only if P0 is stable. **Deferred** is roadmap only.
- Every cross-person interface lives in `contracts/` (Section 14). Never change a contract silently.

1. Assumptions (confirm at kickoff)

- **A1** Hackathon length is **24 hours**. If it is longer, restore deferred items in the order of Section 17.
- **A2** We ship a mobile-first PWA.
- **A3** Languages: Hindi + English (Hinglish input understood).
- **A4** Demo region: Delhi-NCR + Uttar Pradesh mandis. Demo crops: **Tomato and Potato** (forecast and disease), Onion, Wheat and Maize appear only in price browsing if data is available.
- **A5** Buyers are simulated and labelled as demo data.
- **A6** Everything runs on free tiers (Gemini, Firebase, Supabase, Vercel, Hugging Face Spaces). Re-check limits in hour 0, including the Spaces CPU RAM and sleep policy.
- **A7** Each person uses Claude Code inside their own workspace folder.

2. Problem, vision and goals

Problem. A farmer's real question is "what should I do now?", but tools answer slices: weather, mandi prices, disease photos, a separate marketplace. The farmer joins the dots alone, often in a language the app does not speak well, without knowing whether a price is fair or when to sell.

Vision. "Take a photo or say a sentence. KisanSetu tells you what is wrong, what to do today, when and where to sell, and who will pay a fair price."

Goals.

- One joined-up action plan instead of five answers.
- ML that is visible and trustworthy: confidence, heatmaps, backtests, sources.
- A concrete farmer-to-buyer link using a fairness benchmark.
- Works on a cheap Android phone in Hindi.

Non-goals (say these out loud to judges). Payments, logistics, real buyer KYC, IoT, drones or satellite imagery, offline ML inference, any claim of replacing an agronomist.

2.5 Success metrics (24-hour targets, shown on a slide)

- Disease top-1 accuracy on the PlantVillage-style validation set: ≥ 95%.
- Disease accuracy on PlantDoc field-style test images: report the honest number; aim ≥ 60%.
- Price forecast MAE vs naive "last price": beat it by ≥ 10% on at least 1 of the 2 forecast commodities (others show "Trend only").
- P10–P90 band coverage: 70–90%.

- Agent tool-routing accuracy on a **20-question** golden set: ⩾ 90%.
- Uncited dosage statements in the golden set: 0.
- Agent latency p50: text < 8 s, image < 12 s.
- Full demo flow completes 3 runs in a row.

3. Users and personas

- **Ramesh, 42, small farmer** (3 acres, UP). Android, patchy 4G, comfortable with voice notes and WhatsApp, not long forms. Wants: "Is this disease serious? Is today a good day to sell?" **Primary demo persona.**
- **Imran, 35, small trader** (Ghaziabad). Wants nearby farmers selling tomato or potato at a workable price. Appears in the marketplace scene.
- **Sunita, 50, FPO coordinator.** Future channel for scale (Section 23).

4. Landscape and positioning

KisanSetu is a **decision layer on top of public data**, not a competitor to e-NAM.

- **e-NAM:** national online mandi trading. Gap: mandi-bound, no personal advice or sell timing.
- **Kisan e-Mitra / Kisan Sarathi:** government multilingual advisory. Gap: answers questions but does not combine your photo, weather, prices and buyers.
- **Agmarknet:** daily prices. Gap: raw numbers, no forecast, no "sell or wait".
- **Disease-photo apps:** stop at diagnosis, no weather timing, no market link.
- **DeHaat, AgroStar:** commercial platforms built around their own supply chains.

Our edge: cross-tool reasoning (disease + weather + price + buyer) in a Hindi agent, with explainable ML.

5. Product concept and user journeys

- **J1 Scan and act (hero).** Photo of a tomato leaf → disease with heatmap and confidence → treatment steps with sources → "rain expected in 2 days, do not spray tomorrow" → "tomato prices likely to rise, hold 4 days".
- **J2 Ask by voice (P1).** "Mere tamatar ka rate kaisa rahega?" → agent reads prices and forecast → reply in Hindi, optionally read aloud.
- **J3 Sell smart.** Create a listing → ranked buyers with fair-price badge → accept the best bid.
- **J4 Buyer bids.** Imran browses open listings, bids, sees status change when accepted.

6. Feature catalogue

- **F1** Auth and onboarding (Firebase) — **P0** — Arpit
- **F2** Disease scan with explainability — **P0** — Aryan (model, scan UI), Utsav (treatment RAG)
- **F3** Price intelligence and sell-timing — **P0** — Arpit (model, UI), Utsav (agent tool)
- **F4** KisanSetu Agent chat (text + image **P0**; voice and Listen **P1**) — Utsav
- **F5** Marketplace: listings, matching, bids, deals — **P0** — Utsav (matching), Arpit (UI, DB)
- **F6** Home dashboard — **P0** — Arpit (uses a **static Top Action** in this build)
- **F7** Crop recommendation with SHAP — **Deferred**
- **F8** Irrigation planner — **Deferred**
- **F9** Scheme assistant — **Deferred**
- **F10** Notifications — **Deferred**
- **F11** Produce quality grading — **Deferred**
- **F12** Offline-lite — **Deferred** (price snapshots are still cached)

F1 — Auth and onboarding (P0)

Google or phone OTP via Firebase. 3-step onboarding: role (Farmer / Buyer), language (Hindi / English), location (state, district) and, for farmers, one farm (main crop, area, soil type). Role is stored in the database, not in token claims. Phone OTP is primary; use Firebase **test phone numbers** so no real SMS is needed. **Acceptance:** new user reaches Home in under 90 s; session survives refresh; farmers and buyers see different home screens.

F2 — Disease scan (P0)

Input: camera or gallery image, client-compressed to max 1024 px and under 500 KB. Pipeline: upload to private storage → ML module returns top-3 classes, calibrated confidence and Grad-CAM heatmap → confidence gate → treatment retrieval from the RAG corpus → result saved.

- **Confidence ≥ 0.80:** show diagnosis as likely.
- **0.50–0.80:** "Likely, please verify" plus a Gemini vision second opinion shown separately.
- **< 0.50:** "Not sure. Retake the photo" with photo tips; Gemini attempt shown as low confidence.

Thresholds are starting values; Aryan re-tunes them on validation data. Output: disease name (Hindi + English), confidence bar, heatmap toggle, 3–5 treatment steps each with a source chip, and the fixed notice "For chemical dosage, confirm with your local KVK or

agriculture officer." **Acceptance:** result in ⩽ 6 s (p50); scan history; no dosage or chemical quantity without a retrieved source; works from a phone camera on mobile data.

F3 — Price intelligence and sell-timing (P0)

Mandi browser: choose commodity and mandi (default nearest), 30-day chart of modal, min and max price plus arrivals, "prices as of" badge. **Forecast:** 7-day modal price with P10–P90 band, for the 2 forecast commodities. **Sell-timing:** for each mandi within a radius (default 150 km), expected net price = best forecast price in the window minus transport cost (default ₹0.30 per quintal-km, shown as an assumption). Recommend Sell now, Wait N days, or Go to mandi X, with a confidence label. **Honesty rule:** if the model does not beat the naive baseline in backtest, show "Trend only (low confidence)". **Acceptance:** works from the cached snapshot; the demo never calls a live government API; chart and advice render in under 2 s.

F4 — Agent chat (P0 text + image, P1 voice)

Inputs: text and image attach (P0); push-to-talk voice note transcribed server-side (P1). Outputs: streaming reply with visible tool steps ("Checking weather…", "Reading mandi prices…"), then an optional **Action Plan card** (Today / Next 3 days / Selling window, each with a reason and source chips). Listen button uses browser speech synthesis (P1). Replies in the user's language, short sentences. Memory: conversation history per user plus the farm profile. **Guardrails:** max one clarifying question per turn; never invent prices, dosages or amounts; low-confidence disease results flagged; refuse medical or legal advice outside agriculture; always point to KVK or the Kisan Call Centre (1551) for chemical dosage. **Acceptance:** golden-set metrics in 2.5; tool steps stream within 1 s; conversation survives refresh.

F5 — Marketplace (P0)

Farmer: create a listing (crop, variety, quantity in quintal, expected price, available-from date, up to 3 photos, pickup location); see ranked buyer matches with score and reasons; see bids with a **Fair price** badge when the bid is at least 95% of the reference price; accept one bid (deal created, others auto-reject, masked demo contact revealed). **Buyer:** browse open listings (filter by crop and distance), place a bid, track status. **Demo realism:** in this build the marketplace shows **simulated bids only**: a buyer simulator places 2–4 bids from seeded demo buyers within 30–60 s of a new listing, each tagged **Demo buyer**. A second real buyer account may place a bid if time allows.

Matching score (heuristic, weights in config): `score = 0.40 × price_score + 0.25 × proximity_score + 0.20 × reliability + 0.15 × quantity_fit`, where `price_score = min(bid ÷ reference, 1.15) ÷ 1.15`; reference = forecast P50 for the pickup date

(fallback: latest modal price); `proximity_score = 1 − min(distance_km, 200) ÷ 200`; `quantity_fit = min(buyer capacity, quantity) ÷ quantity`. **Acceptance:** listing → bids → accepted deal in under 3 minutes; reasons visible; deal history saved.

F6 — Home dashboard (P0)

Greeting, today's weather and 3-day rain outlook (Open-Meteo), a **static Top Action card** (agent-generated daily plan is deferred), price strip for the farm's crops, and four big buttons: Scan leaf · Ask · Prices · Sell.

Deferred features (roadmap specs kept short)

- **F7 Crop recommender:** RF/XGBoost on Kaggle Crop Recommendation + ICRISAT district data, SHAP reasons, re-rank 0.6 × model + 0.4 × district suitability, saved as joblib.
- **F8 Irrigation planner:** rule-based FAO-56 water balance (ETc = ET0 × Kc, irrigate at 50% depletion, skip if ≥ 8 mm rain at ≥ 60% probability in 24 h). No ML, because no good Indian field dataset exists.
- **F9 Scheme assistant:** RAG over PM-KISAN, PMFBY and KCC documents, quoting source and date.
- **F10–F12:** alerts, produce grading, offline-lite.

Non-functional requirements

- **Performance:** first meaningful screen < 3 s on 4G; images compressed client-side.
- **Accessibility:** tap targets ≥ 48 px, base font 16–18 px, high-contrast light theme, icons plus text, voice where built.
- **Security:** Firebase ID token verified on every API call; leaf images in private storage; no secrets in the frontend; users can delete scans.
- **Reliability:** cached data; every external call has a timeout and fallback message; Space is warmed up before the demo.
- **Observability:** LangSmith traces for every agent run; structured logs.

7. AI/ML specifications

7.1 Disease classifier (Aryan) — F2

- **Data:** New Plant Diseases (Kaggle, ~87K augmented images, 38 classes) plus PlantDoc (~2.6K field images). Lab-background images alone fail on field photos.

- **Model:** EfficientNet-B0 or MobileNetV3-Large via `timm`, transfer learning (a few epochs frozen, then unfreeze), AdamW, label smoothing, mixed precision. **Trained on Colab or Kaggle GPU.** Given 24 hours, restrict training to the demo crops (tomato, potato, plus others only if the field test is acceptable) and cap training time.
- **Field-gap work:** background augmentation, colour jitter, blur, random erasing, PlantDoc mixed in with weighted sampling.
- **Evaluation (report both):** top-1 and macro-F1 on a validation split; separate field test on held-out PlantDoc; confusion matrix.
- **Calibration:** temperature scaling on validation data.
- **Explainability:** Grad-CAM overlay returned as PNG.
- **Artefact format:** export to **ONNX** (`disease.onnx`) plus `labels.json` (class id, English name, Hindi name, crop code) and `metrics.json`. **Do not pickle the PyTorch model.** The backend needs only `onnxruntime`, not `torch`. For Grad-CAM, either run it offline for the demo images and in the backend using an ONNX-compatible approach, or export the last conv feature map as a second ONNX output. Aryan decides in hour 1–6 and records the choice in his CLAUDE.md.
- **Serving:** CPU target < 150 ms per image at 224 px.

7.2 Price forecaster (Arpit) — F3

- **Data:** Agmarknet daily min/max/modal price and arrivals via data.gov.in or the CEDA mirror, **downloaded and cached before the hackathon** (the live API is often down). At least 2 years where available. Aryan owns the ETL.
- **Cleaning:** forward-fill gaps up to 3 days, otherwise mark missing; drop mandis with too little history.
- **Features:** lags (1, 3, 7, 14, 28), rolling mean and std, arrivals, day of week, month, mandi id, simple festival flag.
- **Model:** LightGBM, one model per commodity across mandis, horizons 1, 3 and 7 days, quantile objectives P10 / P50 / P90, for **Tomato and Potato only**. Trained in Colab and saved as joblib.
- **Evaluation:** 60-day walk-forward; MAE and MAPE vs last-price naive and seasonal naive; band coverage; saved to `metrics.json` and shown in the app.
- **Serving:** a one-off script (re-runnable, no nightly cron in 24 h) writes forecasts to `price_forecasts`; the app and agent only read the table. No live inference, no cold-start risk.

7.3 Environment and version rules for model files

- **Pin versions.** Colab's `sklearn`, `lightgbm`, `onnx` and `onnxruntime` versions must match `requirements.txt`. Save `pip freeze` from Colab and hand it to Arpit in hour 0–1.
- **Only load your own pickle/joblib files.** Never open one downloaded from elsewhere.
- **Models are never in git.** Upload to Hugging Face Hub (or the Space's Git LFS) and download at startup. Load each model **once at startup**, not per request.

7.4 Data catalogue

- New Plant Diseases (Kaggle) — F2 training — Aryan.
- PlantDoc — F2 field test and training mix — Aryan.
- Agmarknet (data.gov.in / CEDA) — F3 — Aryan (ETL), Arpit (model).
- Open-Meteo — F6 weather and agent tool — Utsav. Free, no key, attribution required, non-commercial.
- ICAR / KVK public PDFs — RAG — Utsav. Store source and date.
- Deferred: Kaggle Crop Recommendation, ICRISAT, NASA POWER.

8. Agent system specification (owner: Utsav)

A LangGraph graph inside the combined FastAPI backend, Gemini Flash as the reasoning model, Pydantic structured output for the plan, LangSmith tracing.

```
user input (text | image [| audio])
  → [intake]    detect language, extract crop/location/intent
  → [router]    picks tools: diagnose | weather | price | market
  → tool nodes (parallel where independent): diagnose_tool, weather_tool,
                price_tool, match_tool, treatment_rag
  → [planner]   merge into ActionPlan (JSON)
  → [guardrail] cite check, confidence check, dosage check
  → [responder] Hindi/English, short, TTS-friendly
```

State: messages, language, farmer_profile, farm, crop, image_ref, tool_results (with timestamp and source), plan, flags (low_confidence, missing_info, blocked_reason).

Tools (P0):

- `diagnose_tool` — **direct function call** into the ML module (`ml_service.inference`); returns top-3, confidence, heatmap.
- `treatment_rag` — pgvector search filtered by crop and disease; returns steps with source, page, date.

- `weather_tool` — Open-Meteo, cached by rounded lat/lng; returns 7-day forecast and rain probability.
- `price_tool` — reads `mandi_prices`, `price_forecasts`; returns trend, band, sell-timing advice (shared function with core_api's price advice service).
- `match_tool` — reads listings, buyers, bids; returns ranked buyers with reasons.
- Deferred: irrigation_tool, crop_tool, scheme_rag.

RAG. Supabase pgvector. Corpus: about **10 public ICAR / state / KVK crop-protection documents** for tomato and potato (record URL, title, retrieval date). Chunks of about 800 tokens with overlap; metadata crop, disease, source, page, lang. Retrieval: metadata filter then vector search. If nothing relevant is found, say so and refer to the KVK or Kisan Call Centre (1551).

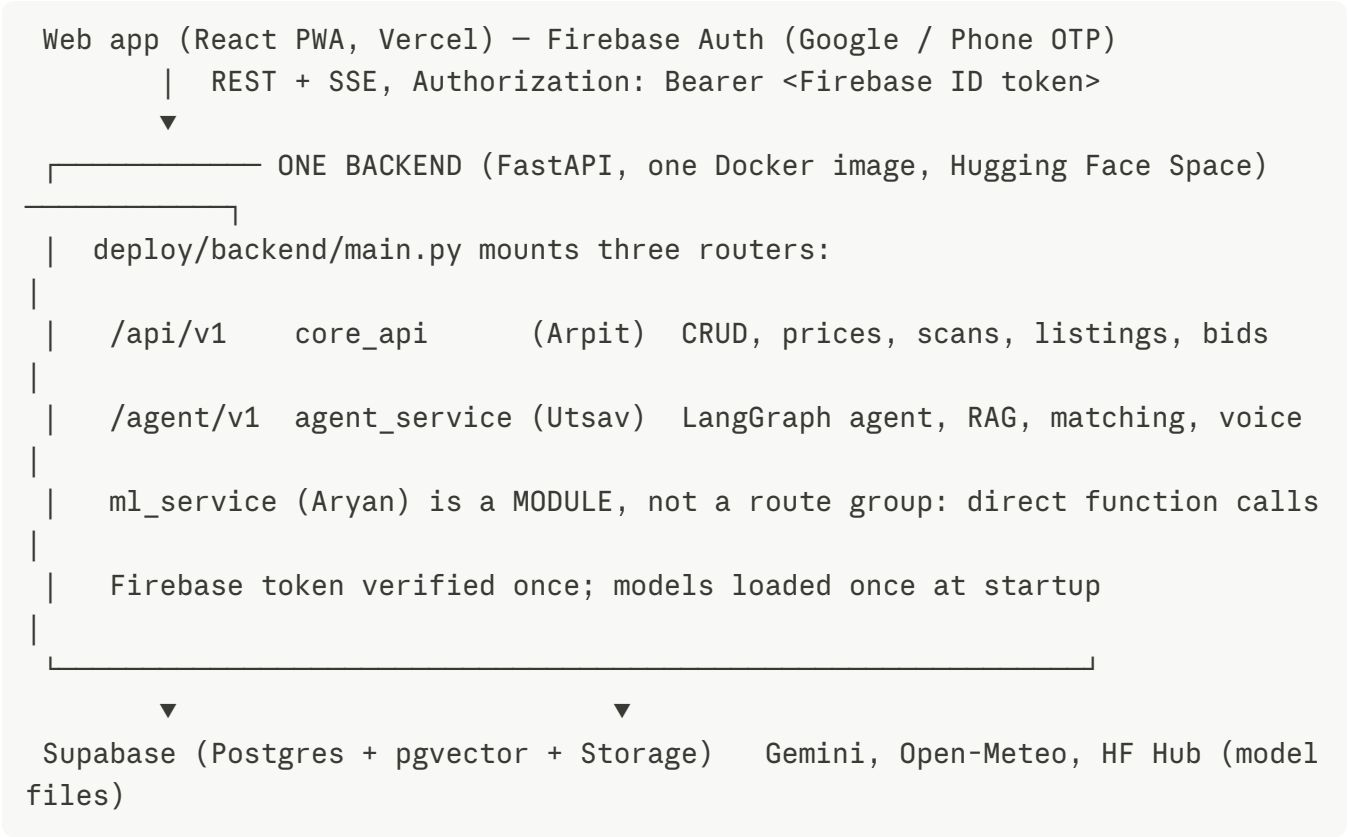
Guardrails (enforced in the guardrail node, not only the prompt):

- Any sentence with a dosage, quantity or amount must map to a retrieved chunk; otherwise it is removed and replaced with the referral line.
- Disease confidence below the F2 gate forces "verify" wording.
- Prices and forecasts carry their date and confidence label.
- Out-of-scope requests get a polite redirect.
- A tool failure produces an honest "could not fetch" message, never a guess.

Evaluation. 20-question golden set (Hindi, Hinglish, English) with expected tool routes and must-cite flags. Metrics: tool-routing accuracy, citation rate, language match, p50 latency. Run before IC2 and IC3.

Voice (P1). Browser MediaRecorder → `POST /agent/v1/transcribe` → Gemini audio understanding returns text and language. TTS uses the browser's speech synthesis.

9. System architecture (combined backend)



Why combined: one cold start, one environment set, CORS in one place, no internal HTTP calls, only one thing to warm before the demo. **Folders stay separate** so code ownership is unchanged.

How it is wired.

- `deploy/backend/` (owner: Arpit) holds `main.py`, the combined `requirements.txt` and the `Dockerfile`. `main.py` imports `core_api.main.router`, `agent_service.main.router` and initialises `ml_service` at startup. Nothing else lives there.
- **Package rename (hour 0):** each workspace's package is renamed from `app` to `core_api`, `agent_service` and `ml_service` so imports do not collide.
- **Local development:** each person can still run their own workspace alone (`uvicorn core_api.main:app`, etc.) against mocks. Nobody depends on anybody else locally.
- The ML module exposes plain functions: `predict_disease(image_bytes, crop_hint) -> DiseaseResult` and `model_info()`. **The ML function signatures are the contract** (see Section 13). `core_api` and `agent_service` call it through a thin wrapper, `ml_client.py`, which today calls the function directly and could later call a remote URL (`ML_SERVICE_URL`) without changing callers.
- Heavy inference runs in a thread pool (`run_in_threadpool`) so it does not block the event loop.

Who calls whom. The web app calls `/api/v1` for everything except chat and `/agent/v1` for chat (and transcription if built). `core_api` calls the ML module directly for scans. `agent_service` calls the ML module directly for diagnosis and reads Supabase with the service role. The ML module is never exposed to the browser. The browser may read public tables only (districts, mandis, mandi_prices, price_forecasts) with the anon key under read-only RLS.

Trade-offs we accept. One bug can take down the whole backend, and a rebuild is slower. Mitigation: dependencies sit in their own Docker layer so code-only rebuilds are fast. **Fallback:** if dependency conflicts (LangChain vs ONNX/NumPy pins) cannot be solved, move the ML module to a second Space and set `ML_SERVICE_URL`. Contracts do not change.

Data flow for J1. Web uploads to `/api/v1/scans` → image stored in the private bucket → `predict_disease` runs in-process → scan saved and returned → UI offers "Get my action plan" → `/agent/v1/chat` with the scan id → agent runs `treatment_rag`, `weather_tool`, `price_tool` → plan merged, guardrail runs, reply streams.

10. Tech stack (decided)

- **Frontend:** React 18 + Vite + TypeScript, Tailwind, shadcn/ui, TanStack Query, react-i18next, Recharts, vite-plugin-pwa. (Leaflet map is optional.)
- **Auth:** Firebase Authentication (Google + Phone OTP); Firebase Admin SDK in the backend.
- **Database and storage:** Supabase Postgres + pgvector + Storage.
- **Backend:** Python 3.11, FastAPI, Pydantic v2, SQLAlchemy 2; LangGraph, LangChain core, `langchain-google-genai`, Gemini Flash, LangSmith.
- **ML:** training in PyTorch + `timm` + albumentations on Colab; serving with **ONNX Runtime**, scikit-learn / LightGBM (joblib). `torch` is a training dependency only, **not** a backend dependency.
- **Training compute:** Google Colab or Kaggle free GPU.
- **Hosting:** Vercel (web), **Hugging Face Spaces with Docker (combined backend)**. Render is no longer used.
- **Tooling:** Claude Code per workspace, `ruff` + `pre-commit`, GitHub.

Free-tier gotchas: Spaces may sleep when idle (warm up 10 minutes before the demo); Supabase free projects pause after inactivity (open the project daily); Gemini free tier has rate limits (cache aggressively, keep tool calls few).

11. Database design (Supabase Postgres)

One schema, owned by Arpit, in `workspace-app/supabase/migrations/`. Others request changes by PR. User ids are Firebase UIDs stored as text. Timestamps are `timestamptz` (UTC). Prices are numeric, rupees per quintal. pgvector is enabled in the first migration. Table and column names are the contract; SQL is generated during the build.

Tables kept for this build

- **Reference and market data (browser read-only):** `districts` (id, state, district, lat, lng); `mandis` (id, state, district, name, lat, lng, agmarknet_code; about 10 for the demo); `diseases` (id, crop_code, class_label, name_en, name_hi; `class_label` must match `labels.json`); `mandi_prices` (price_date, mandi_id, commodity, variety, min_price, max_price, modal_price, arrivals_qtl; unique on date, mandi, commodity, variety); `price_forecasts` (mandi_id, commodity, issued_on, target_date, horizon_days, p10, p50, p90, model_version); `price_model_metrics` (commodity, evaluated_on, mae, mape, naive_mae, band_coverage, verdict = beats_baseline or trend_only; drives the F3 honesty rule).
- **Users and farms (backend only):** `users` (uid PK, role farmer / buyer / admin, display_name, phone, email, language, district_id); `farms` (id, user_uid, name, district_id, lat, lng, area_acres, soil_type, irrigation_type, main_crop_code); the separate `farm_crops` table is deferred.
- **AI tables:** `disease_scans` (id, user_uid, farm_id, image_path, crop_hint, top3 jsonb, top_label, confidence, gate, heatmap_path, second_opinion jsonb, model_version); `conversations`; `messages` (role, content, attachments, plan jsonb, tool_trace jsonb, lang); `documents` (title, publisher, source_url, doc_type, crop_code, lang, retrieved_on); `doc_chunks` (document_id, content, page, crop_code, disease_label, embedding vector, metadata); `weather_cache` (cache_key, fetched_at, payload; 3-hour freshness).
- **Marketplace:** `buyers` (user_uid nullable, name, buyer_type, district_id, lat, lng, crops text[], capacity_qtl, reliability 0–1, is_demo); `listings` (farmer_uid, farm_id, crop_code, variety, quantity_qtl, expected_price, available_from, image_paths, lat, lng, reference_price snapshot, status open / matched / sold / expired); `bids` (listing_id, buyer_id, price_per_qtl, quantity_qtl, message, match_score, fair_price bool, status pending / accepted / rejected / withdrawn); `deals` (listing_id unique, bid_id, final_price, quantity_qtl, status).
- **Deferred tables:** `crops` (Kc values), `farm_crops`, `agent_plans` (daily plan cache).

Relationships. users 1—* farms; users 1—* disease_scans; users 1—* conversations 1—* messages; users 1—* listings 1—* bids —*1 buyers; listings 1—1 deals; documents 1—* doc_chunks; mandis 1—* mandi_prices / price_forecasts.

Access control. Row Level Security is on for every table. The browser (anon key) gets read-only policies on the reference and market tables only. Everything else has no anon access. The backend uses the service-role key after verifying the Firebase token itself (Supabase's `auth.uid()` expects Supabase-issued JWTs, and our users sign in with Firebase). The service role key lives only in backend environment variables.

Storage buckets. `leaf-images` (private, JPG/PNG, max 5 MB, signed URLs); `heatmaps` (private, written by the backend); `listing-images` (public read, max 3 per listing, 5 MB each).

Seed data (Arpit, Aryan helps), required before IC1. About 15 districts (Delhi, UP, Haryana), about 10 mandis, at least 2 years of cached prices for Tomato and Potato (and Onion, Wheat, Maize if the data exists), 20 demo buyers, 3 demo farmers with farms, first RAG chunks. Seed scripts live in `workspace-app/supabase/seed/` and are re-runnable.

12. Authentication (Firebase)

Providers: Google OAuth and Phone OTP (phone primary for farmers).

1. The web app signs in with the Firebase JS SDK and gets an ID token.
2. Every call sends `Authorization: Bearer <token>`.
3. The backend verifies the token with the Firebase Admin SDK (service account JSON, base64 in an env var). **One verification module** (`auth/firebase.py`) lives in the combined backend as a shared FastAPI dependency; each workspace imports it or keeps its own copy, but there is a single verification pass per request.
4. On first call `POST /api/v1/auth/session` upserts the `users` row. Role is set in onboarding and read from the database later.
5. **There is no service-to-service key any more.** Core API, agent and ML code call each other as in-process functions, passing the already-verified user uid as an argument.

Setup checklist (Arpit, hour 0–1). Create the Firebase project; enable Google and Phone; add `localhost`, the Vercel domain and the preview domain to Authorized domains; add **test phone numbers** with fixed codes; generate the service account key and share it privately (never git); configure CORS for the web origins only (one place now).

PWA caveat. Google sign-in popups can be flaky in an installed iOS PWA. Test on a real phone early, keep Phone OTP primary, use the redirect flow if needed.

13. API contracts

OpenAPI files live in `contracts/openapi/` (`core-api.yaml`, `agent-service.yaml`). All routes except `/health` need the Firebase bearer token. Errors use one shape: `{ code, message, details? }`.

13.1 core_api — `/api/v1` (Arpit)

- `POST /auth/session`; `GET/PATCH /me`
- `GET/POST/PATCH /farms`
- `POST /scans` (multipart image), `GET /scans`, `GET/DELETE /scans/{id}` → DiseaseResult
- `GET /mandis?district_id=`; `GET /prices?commodity=&mandi_id=&days=30`; `GET /prices/forecast?commodity=&mandi_id=`; `GET /prices/advice?commodity=&farm_id=&quantity_qtl=` → PriceAdvice
- `POST/GET /listings`, `GET /listings/{id}`; `GET /listings/{id}/matches` (calls the matching function in-process)
- `POST/GET /listings/{id}/bids`; `POST /bids/{id}/accept` → Deal; `GET /deals`
- `POST /uploads/sign`

13.2 agent_service — `/agent/v1` (Utsav)

- `POST /chat` (SSE): input message, optional `image_path` or `scan_id`, `lang`; events `step`, `token`, `plan`, `done`, `error`.
- `GET /conversations`, `GET /conversations/{id}/messages`.
- `POST /transcribe` (P1).
- Deferred: `POST /daily-plan`.
- The old `POST /internal/match` is removed; matching is a Python function (`agent_service.matching.rank_buyers(listing, buyers)`) called by core_api.

13.3 ml_service — Python function contract (Aryan), not HTTP

- `predict_disease(image_bytes: bytes, crop_hint: str | None) -> DiseaseResult` (without DB ids)
- `model_info() -> dict` (model versions and metrics)
- Startup hook `load_models()` loads `disease.onnx`, `labels.json` and the price joblib (if the backend serves price inference) **once**.
- Deferred: `predict_crop`, `predict_quality`.

- The wrapper `ml_client.py` is the only place that calls these functions, so swapping to a remote `ML_SERVICE_URL` later changes one file.

13.4 Shared object shapes (JSON Schemas in `contracts/schemas/`)

- **DiseaseResult:** top3 [{label, name_en, name_hi, confidence}], top_label, confidence, gate (confident / verify / retake), heatmap (URL or base64 PNG), model_version, second_opinion (object or null; Gemini result only when gate is not confident).
- **PriceAdvice:** recommendation (sell_now / wait / go_to_mandi), wait_days, target_mandi {id, name, distance_km}, expected_net_price, forecast_band {p10, p50, p90}, confidence (high / medium / low / trend_only), assumptions[], data_as_of.
- **ActionPlan:** language, summary, actions[] {when: today / next_3_days / selling_window, title, reason, priority, sources[]}, warnings[], confidence, data_as_of {prices, weather}. sources[] = {label, url, date}.
- **MatchResult:** buyer {id, name, type, distance_km, reliability, is_demo}, score 0–1, reasons[], fair_price boolean.

14. Repository and folder structure

One monorepo, three workspaces plus shared `contracts/`, `docs/`, `infra/` and the new `deploy/backend/`. You edit only your own workspace. Contracts change by pull request with both teammates tagged.

```
kisansetu/
├── README.md, CLAUDE.md, .env.example
├── docs/                    PRD.md, demo-script.md, judge-qa.md
├── contracts/               SHARED: openapi/, schemas/, examples/ (mock
payloads)
├── deploy/backend/          OWNER ARPIT: main.py (mounts routers),
requirements.txt,
|                            Dockerfile (deps layer separate), start.sh
(model download)
├── infra/                   docker-compose.yml (local), warmup.sh
├── workspace-agent/         UTSAV
|   └── agent_service/       package renamed from app/
|       ├── main.py (exposes router), config.py
|       ├── api/ (chat, transcribe, conversations)
|       ├── graph/ (state, builder,
nodes/{intake,router,planner,guardrail,responder})
|       ├── tools/ (diagnose, treatment_rag, weather, price, match)
|       ├── matching/ (scoring.py, config.yaml)
|       ├── rag/, llm/ (prompts), schemas/, db/
|   ├── rag_corpus/ (raw_pdfs, manifest.csv), evals/ (golden_set.jsonl,
```

```
run_evals.py)
├── workspace-ml/            ARYAN
|   ├── notebooks/           Colab: disease EDA, train, field eval
|   ├── training/disease/    dataset, augment, train, calibrate, export_onnx
|   ├── ml_service/          package renamed from app/
|   |   ├── loader.py        load_models() at startup
|   |   ├── inference/       disease_onnx.py, gradcam.py
|   |   ├── api.py           predict_disease(), model_info()  (the contract)
|   |   └── models/          NOT in git: disease.onnx, labels.json,
metrics.json
|   ├── scripts/             buyer_simulator.py, make_demo_images.py
|   └── reports/             metrics, confusion matrix
└── workspace-app/           ARPIT
    ├── web/                 React PWA (src/lib, pages, components, locales,
mocks)
    ├── core_api/            package renamed from app/
    |   ├── main.py (exposes router), config.py
    |   ├── api/ (auth, me, farms, scans, prices, mandis, listings, bids,
uploads)
    |   ├── services/ (price_advice.py, ml_client.py, storage.py)
    |   └── db/, schemas/
    ├── pricing/             etl/ (Aryan), train/ (features, train_lgbm,
backtest), jobs/ (forecast script)
    └── supabase/            migrations/, seed/
```

There is no `ml-service/Dockerfile` or `requirements.txt` any more; `deploy/backend/` owns the single image. Each workspace keeps a small `requirements.in` listing its own dependencies so Arpit can merge them into one file.

15. Working agreements

- **15.1 Ownership.** One owner per folder. For others' folders, ask in chat. Shared writable areas: `contracts/` (by PR) and `docs/`. `deploy/backend/main.py` belongs to Arpit; the others only expose a `router` (or an init function for ML).
- **15.2 Mock-first.** Hour 0–1 output: `contracts/examples/` holds one realistic payload per endpoint, shape and ML function. Each person works on mocks until the next checkpoint; at each checkpoint a mock is switched for the real module. Nobody waits.
- **15.3 Git rules.** Branches `agent/<topic>`, `ml/<topic>`, `app/<topic>`; `main` always runs; merge small PRs at least every 3 hours; squash merge. **Never commit secrets, datasets or model weights.** Weights go to Hugging Face Hub; `start.sh` downloads them at container start. Contract changes are a separate PR labelled `contract`.

- **15.4 Claude Code operating model.** Each workspace has a `CLAUDE.md` with purpose, stack, run/test/lint commands, editable folders, contracts, definition of done and a don't list. Pattern: Spec → Plan → Implement → Test → Commit. Use Claude Code for scaffolding, CRUD, Dockerfiles, Pydantic mirrors, i18n, seed scripts. Keep for humans: model and threshold choices, prompt wording, guardrail rules, the demo script.
- **15.5 Environment variables (names only).** `FIREBASE_PROJECT_ID`, `FIREBASE_SERVICE_ACCOUNT_B64` (backend); `VITE_FIREBASE_*` (web, public by design); `SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` (web, read-only RLS); `SUPABASE_SERVICE_ROLE_KEY`, `DATABASE_URL` (backend only, never in web); `GOOGLE_API_KEY` (Gemini); `LANGSMITH_API_KEY`, `LANGSMITH_TRACING`; `HF_TOKEN`, `HF_MODEL_REPO` (to download model files); `VITE_API_URL` (single backend URL, used for both `/api/v1` and `/agent/v1`). **Removed:** `INTERNAL_API_KEY`, `CORE_API_URL`, `AGENT_SERVICE_URL`, `VITE_CORE_API_URL`, `VITE_AGENT_URL`. **Optional fallback only:** `ML_SERVICE_URL` (empty = call in-process). Secrets are shared privately at hour 0 and stored in the Space's secrets.

16. Work breakdown (24-hour cut)

Each person has about 15 productive hours. Only P0 is scheduled; P1 is attempted after IC2 if stable. Where Sections 6–7 name a different owner, this section wins.

Ownership exceptions. E1: `web/src/components/chat/`, `pages/Chat.tsx`, `lib/sse.ts`, `lib/audio.ts` belong to Utsav. E2: `components/scan/`, `pages/Scan.tsx`, `pages/ScanResult.tsx` belong to Aryan. E3: `pricing/etl/` belongs to Aryan.

16.1 Utsav — Agent, RAG, matching

- U1 (P0, 2 h) Agent skeleton: router export, Firebase dependency, `/health`, SSE `/chat` from mocked tools.
- U2 (P0, 4 h) LangGraph core: state, intake, router, planner (Pydantic ActionPlan), responder, Hindi/English.
- U3 (P0, 1 h) `weather_tool` (Open-Meteo, 3-hour cache).
- U4 (P0, 3 h) RAG: about 10 public tomato/potato documents with manifest, chunk, embed, load pgvector; `treatment_rag`.
- U5 (P0, 3 h) Tool wiring: `diagnose_tool` (direct ML call), `price_tool`, `match_tool` and `rank_buyers()`.
- U6 (P0, 2 h) Guardrail node.
- U7 (P0, 2 h) Chat UI (E1): SSE client, tool-step chips, plan card, image attach.
- U8 (P0, 2 h) Golden set (20 questions), `run_evals.py`, LangSmith project.

- U9 (P1) Voice: `/transcribe`, mic recorder, Listen button.
- Hand-off: by hour 5, `/chat` streams a mocked plan so the UI can integrate. Done when routing ≥ 90%, zero uncited dosage statements, and every tool failure shows an honest message.

16.2 Aryan — Disease model, ML module, scan UI, price ETL

- A1 (P0, 2 h) Colab/Kaggle setup, dataset download, label map with Hindi names; `pip freeze` handed to Arpit.
- A2 (P0, 3 h) Baseline EfficientNet-B0 on the demo crops; first metrics.
- A3 (P0, 4 h) PlantDoc mix, background augmentation, weighted sampling, retrain, field test.
- A4 (P0, 2 h) Temperature scaling, gate thresholds, Grad-CAM, ONNX export, `labels.json`, `metrics.json`; upload to HF Hub.
- A5 (P0, 2 h) `ml_service`: `load_models()`, `predict_disease()`, `model_info()`, unit tests on example images.
- A6 (P0, 3 h) Scan UI (E2): picker, client compression, result screen, heatmap toggle, gate states, history.
- A7 (P0, 3 h) Price ETL (E3): cached Agmarknet data (started before the event), clean, load to Supabase.
- A8 (P0, 1 h) Reports for slides, 6 field examples including 3 honest failures; buyer simulator script.
- Hand-off: by hour 5, a baseline ONNX model on HF Hub and `predict_disease` returning real output. Done when both metrics are reported, gates are checked on 20 unseen photos, and a model swap needs no contract change.

16.3 Arpit — App shell, backend assembly, database, price model

- R1 (P0, 3 h) Monorepo, Vite + React + TS scaffold, Tailwind tokens, routing, i18n, Firebase login, MSW mocks.
- R2 (P0, 3 h) Supabase: migrations, RLS, buckets, seed scripts.
- R3 (P0, 2 h) `core_api` skeleton: auth, `/auth/session`, `/me`, `/farms`, `/uploads/sign`, error shape.
- **R3b (P0, 3 h, hour 0–6) Combined backend: rename packages, merge requirements, `deploy/backend/main.py`, Dockerfile with separate dependency layer, `start.sh` model download, dependency-conflict test at hour 1, first deploy to the Space.**
- R4 (P0, 4 h) Price model for 2 commodities: features, LightGBM quantiles, 60-day backtest, `price_model_metrics`, forecast script.
- R5 (P0, 3 h) `/prices*` endpoints, sell-timing advice, Prices page with chart and band.

- R6 (P0, 1 h) `/scans` (store, in-process ML call, save).
- R7 (P0, 4 h) Marketplace: `/listings`, `/bids`, `/deals`, matches; farmer and buyer screens; simulator hook.
- R8 (P0, 2 h) Onboarding, Home with static Top Action, Profile.
- R9 (P0, 2 h) PWA polish, Vercel deploy, `warmup.sh`, real-phone testing.
- Hand-off: by hour 5, auth, onboarding, empty screens on mocks, migrations applied, and the backend Space running with a `/health` route that lists loaded models. Done when all P0 screens work on a real Android phone over mobile data, RLS is verified, the forecast script has run and the backtest report is committed.

16.4 Cross-cutting

- Contracts and mock examples — Utsav drafts, all review — hour 0–1.
- Seed data — Arpit (Aryan helps) — before IC1.
- Golden-set questions — Utsav, with 7 each from Aryan and Arpit — hour 6–10.
- Slide deck — Utsav leads, each writes their own slide — hour 18–22.
- Real-phone test rounds — everyone — hours 12, 18, 21.
- **Load-balancing rule:** when anyone is blocked or finishes early, announce it and take the top item from another backlog. Aryan's first overflow target is Arpit's marketplace screens; Arpit's is Utsav's weather tool and RAG ingest.

17. Timeline (24-hour plan)

- **Hour 0–1 — Kickoff.** Accounts (Firebase, Supabase, Gemini, LangSmith, Hugging Face), repo, three CLAUDE.md files, contracts and mock examples, secrets shared, dataset and Agmarknet downloads started, **dependency-conflict test of the merged requirements**.
- **Hour 1–6 — Slice 1: skeletons on mocks.** Auth works; baseline disease ONNX on HF Hub; agent streams a mocked plan; DB migrated and seeded; backend Space deployed. **IC1 (hour 6):** each module answers a real request.
- **Hour 6–16 — Slice 2: real tools.** Field-aware model; price model and backtest; RAG ingested; marketplace flow; Home. **IC2 (hour 16):** J1 and J3 work end to end on the deployed backend (rough is fine).
- **Hour 16–20 — Polish.** Evals and guardrails, bug fixing, voice or P1 only if P0 is stable, demo data. **IC3 (hour 20): FEATURE FREEZE.**
- **Hour 20–24 — Demo, deck, rehearsal.** Warm-up script, backup video, three timed rehearsals, submission with buffer.

- **Sleep:** one 3-hour block each between hours 11 and 17, staggered so two people are always awake. Nobody sleeps through a checkpoint.
- **Cut list (already applied):** F7–F12, crop model, scheme RAG, irrigation, daily plan (static Top Action), voice (P1), forecasts for 2 commodities, RAG corpus of about 10 documents, simulated bids in the marketplace. If time is lost, cut in this order: voice, buyer-side real bidding, history screens, second forecast commodity.

18. UI/UX specification

Principles. Thumb-first, sunlight-readable, one primary action per screen, plain Hindi before English jargon, every number shows its date and confidence.

Colour theme "Mitti & Hariyali". Primary Leaf Green `#2E7D32`; primary-dark `#1B5E20`; primary-tint `#E8F5E9`; accent Harvest Gold `#F9A825` (dark text on it); secondary Soil Brown `#6D4C41`; background Cream `#FAF7F0`; surface White; border `#E3DCCB`; text `#1C2B1F`; muted `#5B6B5E`; water `#1E88E5`; warning `#EF6C00`; danger `#C62828`. Light theme only. Semantic mapping: green = good or confident, gold = highlight or fair price, orange = verify, red = problem, blue = weather and water. Aim for WCAG AA contrast.

Typography and shape. Headings Poppins 600/700; body Noto Sans with Noto Sans Devanagari. Body 16–18 px; H1 28; H2 22; buttons 18. Card radius 16 px, button radius 12 px, 4 px grid, tap targets ≥ 48 px. Icons from Lucide.

Layout. Mobile-first at 360–412 px. Farmer bottom tabs: Home · Scan · Ask · Prices · Sell. Buyer tabs: Home · Listings · My Bids · Profile. On screens ≥ 1024 px: left sidebar and a centred 560 px column. Language toggle (हि/EN) always in the header.

Screens. Landing `/`; Login `/login`; Onboarding `/onboarding`; Home `/home`; Scan `/scan`; Scan result `/scan/:id`; Prices `/prices`; Ask `/chat`; Sell `/sell`; Listing detail `/listings/:id`; Buyer home `/buyer`; Buyer listings `/buyer/listings`; Profile `/profile`.

Interaction rules. Skeleton loaders instead of spinners; tool-step chips animate during agent runs; badges: Fair price (gold), Demo buyer (grey with gold outline), Trend only (grey); friendly empty states; human-sentence errors with a retry button.

19. Website to phone app (PWA)

Build a PWA now (2–3 hours if the layout is mobile-first). Capacitor APK is optional after the hackathon; React Native or Flutter is out of scope.

Checklist (Arpit). `manifest.webmanifest` (name, 192 and 512 icons including maskable, `display: standalone`, `start_url`, theme `#2E7D32`); service worker via `vite-plugin-pwa` (precache shell, stale-while-revalidate for price GETs, `offline.html`); HTTPS and safe-area padding; camera via a file input opening the rear camera; microphone via MediaRecorder (HTTPS and a tap; only if voice is built); Android install button; iOS "Share → Add to Home Screen" hint; skip push on iOS; test every release on a real phone via the Vercel preview URL.

20. Deployment and operations

- **Web:** Vercel. Env: `VITE_*`; add the domain to Firebase Authorized domains.
- **Backend (combined): Hugging Face Space, Docker SDK**, one image, one URL. Hugging Face Spaces gives more free RAM than Render's 512 MB, which matters once ONNX Runtime and LangChain share a process. **Verify the current free CPU RAM and sleep policy in hour 0.**
- **Models:** stored on Hugging Face Hub. `start.sh` downloads them at container start (or the Dockerfile bakes them in if the build limit allows) and `load_models()` loads them **once**.
- **Docker layering:** system packages → `pip install -r requirements.txt` (own layer) → application code last, so code-only changes rebuild in seconds.
- **Database, vectors, storage:** Supabase (open the project daily). **Auth:** Firebase test phone numbers.
- **Jobs:** the price forecast script and weather cache are run manually from a laptop (or a scheduled GitHub Action if time allows); no nightly cron in 24 hours.
- **Environments:** local (`docker-compose` or each workspace on its own) and demo (the Space).
- **Demo-day safety net:** (1) `infra/warmup.sh` pings `/health` 10 minutes before the slot; (2) the whole stack runs locally on one laptop with a tunnel as fallback; (3) a screen-recorded backup video; (4) cached price snapshot, so no government API call; (5) hotspot as backup internet.

21. Testing, demo and judging

Test plan. Unit tests (matching score, price advice, dosage guardrail, gate thresholds) by each author; contract tests (example payloads validate against `contracts/schemas/`); agent evals before IC2 and IC3 (Utsav); model eval plus calibration check on 20 unseen photos (Aryan); 60-day backtest (Arpit); end-to-end J1, J3 and J4 on a real Android phone over mobile data (everyone). After the freeze, only P0 bugs are fixed.

Six-minute demo script.

- 0:00 Ramesh's problem in one story (30 s).
- 0:30 J1: photo → diagnosis, heatmap, confidence → action plan (90 s).
- 2:00 Prices: forecast band, backtest, honest "trend only" label on one commodity (60 s).
- 3:00 J3 and J4 on two phones: listing, simulated bids, Fair price badge, accept (75 s).
- 4:15 J2 Hindi voice (45 s) **only if built**; otherwise a Hindi text question in chat.
- 5:00 Under the hood: combined backend, ONNX model, metrics, LangSmith trace (45 s).
- 5:45 Impact and roadmap (15 s).

Rehearse three times with a timer; everyone knows the fallback video's start time.

Likely judge questions.

- *How is this different from e-NAM or Kisan e-Mitra?* They are trading and advisory channels. We are the decision layer on top of the same public data.
- *Is the forecast reliable?* We show a backtest against a naive baseline and a P10–P90 band, and we say "trend only" when we do not beat the baseline.
- *What if the disease model is wrong?* Calibrated confidence gates; low confidence asks for a better photo; medium shows verify plus a second opinion; dosage always defers to KVK.
- *Are the buyers real?* No, they are labelled demo buyers. Onboarding, KYC and payments are the next phase.
- *Why train on Colab and not live?* The models are trained once offline and shipped as files, so the backend only runs fast inference and the demo has no training risk.
- *Will it work on poor networks?* Images are compressed on the device, prices are cached, and the app is an installable PWA.
- *How would it make money?* Buyer-side subscriptions or lead fees, FPO partnerships, lender or insurer integrations, with advice kept neutral.
- *Data privacy?* Private image storage, token-verified APIs, users can delete scans, no secrets in the browser.

Slide outline (8 slides). Problem · Solution and journeys · Live demo cue · How it works · ML results · Positioning · Impact, business model, roadmap · Team.

22. Risks and mitigations

- **Disease model drops on field photos** (High, Aryan): field test, PlantDoc mix, background augmentation, calibrated gates, curated demo photos, honest failures.

- **Government price API down** (High, Aryan and Arpit): cache before the event; demo never calls it.
- **Dependency conflicts in the combined backend** (Medium, Arpit): test in hour 1, pin versions from the Colab `pip freeze`; fallback is to split the ML module onto a second Space via `ML_SERVICE_URL`.
- **One bug takes down the whole backend** (Medium, all): separate dependency layer, small PRs, `/health` check in CI, local fallback.
- **Pickle / ONNX version mismatch** (Medium, Aryan and Arpit): matching pinned versions; load test of every model file in hour 5.
- **Free-tier sleep, pauses or limits** (High, Arpit and Utsav): warm-up script, local fallback, caching, few LLM calls.
- **Gemini latency or quota** (Medium, Utsav): cache weather, stream partial output, keep prompts short.
- **Hallucinated treatment advice** (Medium, Utsav): guardrail node, mandatory citations, KVK referral.
- **Phone OTP or Google sign-in problems** (Medium, Arpit): test numbers, redirect flow, early phone test.
- **Integration slips in 24 hours** (High, all): mock-first, IC1–IC3, load-balancing rule.
- **Scope creep** (High, all): P0/P1/Deferred labels, freeze at hour 20, cut order in Section 17.

Responsible AI. We never present AI output as a prescription. Dosages need sources and a referral to local experts; confidence is always shown; demo data is labelled; farmers' images stay private and deletable.

23. Future scope and business model

- **Scale path:** the deferred features (crop recommender, irrigation planner, scheme assistant, alerts), AgriStack identity and land data, FPO channels, ONDC-style commerce, satellite crop health, more languages, WhatsApp interface.
- **Revenue ideas:** buyer-side subscription or lead fee, FPO and cooperative plans, lender and insurer integrations, clearly labelled sponsored advisories only.
- **Why it scales:** public data plus a reusable agent and model layer; costs are mostly inference, kept low with caching and small ONNX models. If load grows, the ML module can be split into its own service without changing contracts.

Appendix A — Hour 0–1 kickoff checklist

- Repo created with three workspaces, `contracts/`, `deploy/backend/`, `infra/`, `docs/`.
- **Packages renamed** to `core_api`, `agent_service`, `ml_service`.
- Three `CLAUDE.md` files written from Section 15.4.
- `contracts/examples/` has one payload per endpoint, shape and ML function.
- **Merged requirements installed in a clean environment; import test passes (LangChain + ONNX Runtime + LightGBM + FastAPI).**
- Colab `pip freeze` saved and shared.
- Firebase (Google + Phone, test numbers), Supabase, Gemini key, LangSmith key, Hugging Face account and model repo created.
- Secrets shared privately; `.env.example` committed.
- Datasets and Agmarknet snapshot downloading.
- Free-tier limits re-checked (Space RAM and sleep policy, Gemini, Supabase, Firebase Phone Auth).
- Sleep blocks and checkpoint times agreed.

Appendix B — Release checklist (before hour 20)

- All P0 acceptance criteria in Section 6 pass on a real Android phone.
- Metrics from Section 2.5 recorded and on slides.
- RLS test: anon key cannot read private tables.
- No secrets or model files in git history; `.env.example` complete.
- Space cold-start time measured; `warmup.sh` tested; fallback video recorded; local stack boots with one command.
- Demo accounts and test phone numbers work; demo buyers visible.
- Three timed rehearsals done.

Appendix C — Data and reference sources

PlantDoc (arXiv 1911.10317, github.com/pratikkayal/PlantDoc-Dataset) · PlantVillage and the Kaggle "New Plant Diseases Dataset" · Agmarknet via data.gov.in and the CEDA mirror · Open-Meteo (credit required; free for non-commercial use within fair-use limits). Deferred sources: Kaggle Crop Recommendation, ICRISAT District Level Data, NASA POWER. Licences and API terms must be re-checked before any commercial use.